

CSA05 – Database Management Systems

Assignment

Assignment Title:	Design and Development of a College Library Management Database System
Course Outcome(s) – CO:	CO1: Understand database concepts, architecture, and the need for a DBMS over file-based systems. CO2: Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. CO3: Analyse relational databases using functional dependencies, normalization (1NF–BCNF), and key constraints to identify redundancy and ensure data integrity. CO5: Evaluate transaction handling, concurrency, and query optimization techniques such as indexing for efficient database performance.
Bloom's Taxonomy Level:	L4-Analyze, L5-Evaluate, L6-Create
SDG Mapping (SDG 1–17):	SDG 4 – Quality Education SDG 9 – Industry, Innovation and Infrastructure SDG 16 – Peace, Justice and Strong Institutions

1. Problem Statement

Manual and file-based systems used by many college libraries to record membership, book catalogues, physical copies, and lending activity suffer from significant limitations. Book and member details are often duplicated across multiple registers, making the data inconsistent and difficult to update. Locating whether a specific book is available, tracking which member currently holds a copy, and identifying overdue books require manual searching through paper records or disconnected spreadsheets. There is no reliable mechanism to enforce that a copy cannot be issued to two members at once, that fines are computed correctly, or that a member's contact information stays consistent across records. As the number of members, titles, and daily transactions grows, this manual approach becomes error-prone, slow, and unable to generate reports such as overdue lists or borrowing statistics in a timely manner.

There is, therefore, a need for a relational database-driven College Library Management System that can store Member, Book, Publisher, Copy, and Issue information in a structured, non-redundant form, enforce integrity constraints automatically, and support fast retrieval of information such as overdue books and member-wise borrowing history.

2. Objective

- To identify the entities, attributes, and relationships involved in a College Library Management System and represent them using an Entity–Relationship (ER) diagram.
- To translate the ER model into a relational schema with appropriate primary keys, foreign keys, and constraints.
- To identify functional dependencies among attributes and normalize the relations up to an appropriate normal form (3NF) to eliminate redundancy and anomalies.
- To design and implement the database in a relational DBMS (MySQL) with correct table definitions and referential integrity.
- To write SQL queries that retrieve overdue books and member-wise borrowing statistics.
- To create a view that simplifies repeated access to overdue issue information.
- To recommend and justify indexes on ISBN, Member, and Due Date for efficient query processing.
- To explain how transactions are handled during the book issue and return process to preserve consistency.

3. Requirements and Environment Used

3.1 Functional Requirements

- Maintain records of members, books, publishers, individual book copies, and issue/return transactions.
- Support insertion, updating, deletion, and retrieval of records for all entities.
- Prevent a copy from being issued to more than one member at the same time.
- Automatically identify overdue issues (Due Date passed with no Return Date) and compute fines.
- Generate member-wise and book-wise borrowing reports.

3.2 Non-Functional / Performance Requirements

- Queries on ISBN, Member, and Due Date should execute efficiently even as the Issue table grows large.
- The design should minimize redundant storage of member and book details.
- The system should maintain entity and referential integrity at all times.

3.3 Environment Used

Component	Tool / Technology
RDBMS	MySQL 8.0 (Community Server)
Client / IDE	MySQL Workbench 8.0
ER Diagram / Schema Design	draw.io and MySQL Workbench Reverse Engineering
Operating System	Windows 11 / Ubuntu 22.04
Scripting Language	SQL (DDL, DML, DCL, TCL)
Documentation	Microsoft Word

4. Design / Proposed Solution

4.1 Entities and Relationships

Entity	Key Attributes	Description
Member	Member (PK)	A registered library user who can borrow books.
Book	Bookie (PK), ISBN (Unique)	A distinct book title with its bibliographic details.
Publisher	Publisher (PK)	The publishing house that publishes one or more books.
Copy	Copied (PK), Accession_No (Unique)	A specific physical copy of a Book that can be issued.
Issue	Issue_ID (PK)	A transaction recording the issue and return of a Copy to a Member.

- Publisher – Book: One publisher publishes many books; each book has exactly one publisher (1:N).
- Book – Copy: One book title may have many physical copies; each copy belongs to exactly one book (1:N).
- Member – Issue: One member can have many issue transactions over time; each issue transaction belongs to one member (1:N).
- Copy – Issue: One copy can be issued many times over its lifetime (to different members on different dates); each issue transaction refers to exactly one copy (1:N). Copy and Member therefore share a many-to-many borrowing relationship that is resolved through the Issue associative entity.

5. Algorithm / Pseudocode – Functional Dependencies and Normalization

5.1 Functional Dependencies

Member: Member_ID -> Member_Name, Department_ID, Email, Phone
 Publisher: Publisher_ID -> Publisher_Name, Contact_No
 Book: Book_ID -> ISBN, Title, Author, Publisher_ID, Price
 ISBN -> Title, Author, Publisher_ID, Price (ISBN is a candidate key)
 Copy: Copy_ID -> Book_ID, Accession_No, Status
 Accession_No -> Book_ID, Status (Accession_No is a candidate key)
 Issue: Issue_ID -> Copy_ID, Member_ID, Issue_Date, Due_Date, Return_Date, Fine

5.2 Normalization Steps

1NF: Every attribute in all five relations holds a single atomic value (e.g., Phone and Email are single values per member, and a Copy row represents exactly one physical copy). No repeating groups or multi-valued attributes exist, so all relations satisfy 1NF.

2NF: Every relation has a single-column primary key (Member_ID, Publisher_ID, Book_ID, Copy_ID, Issue_ID), so no non-prime attribute can be partially dependent on part of a composite key. All relations are therefore already in 2NF.

3NF / BCNF: A relation violates 3NF if a non-prime attribute is transitively dependent on the primary key through another non-prime attribute. In this design, Publisher_Name and Contact_No are stored only in Publisher and referenced from Book via Publisher_ID (not duplicated in Book), so there is no transitive dependency of the form Book_ID → Publisher_ID → Publisher_Name inside the Book relation. Similarly, Book details are not duplicated inside Copy, and Member/Copy details are not duplicated inside Issue. Every determinant in each relation (Member_ID, Publisher_ID, Book_ID/ISBN, Copy_ID/Accession_No, Issue_ID) is a candidate key, so all five relations satisfy BCNF, and no further decomposition is required.

5.3 Pseudocode – Fine Computation and Overdue Check

```
FUNCTION isOverdue(issue):
  IF issue.Return_Date IS NULL AND issue.Due_Date < CURRENT_DATE THEN
    RETURN TRUE
  ELSE
    RETURN FALSE

FUNCTION computeFine(issue, ratePerDay = 2):
  end_date = issue.Return_Date IS NULL ? CURRENT_DATE : issue.Return_Date
  overdue_days = MAX(0, DATEDIFF(end_date, issue.Due_Date))
  RETURN overdue_days * ratePerDay
```

6. Implementation / Source Code

6.1 Data Definition Language (DDL)

```
CREATE TABLE Publisher (  
    Publisher_ID INT PRIMARY KEY AUTO_INCREMENT,  
    Publisher_Name VARCHAR(100) NOT NULL,  
    Contact_No VARCHAR(15)  
);  
  
CREATE TABLE Book (  
    Book_ID INT PRIMARY KEY AUTO_INCREMENT,  
    ISBN VARCHAR(20) NOT NULL UNIQUE,  
    Title VARCHAR(150) NOT NULL,  
    Author VARCHAR(100) NOT NULL,  
    Publisher_ID INT,  
    Price DECIMAL(8,2) CHECK (Price >= 0),  
    FOREIGN KEY (Publisher_ID) REFERENCES Publisher(Publisher_ID)  
        ON UPDATE CASCADE ON DELETE SET NULL  
);  
  
CREATE TABLE Member (  
    Member_ID INT PRIMARY KEY AUTO_INCREMENT,  
    Member_Name VARCHAR(100) NOT NULL,  
    Department_ID VARCHAR(10),  
    Email VARCHAR(100) UNIQUE,  
    Phone VARCHAR(15)  
);  
  
CREATE TABLE Copy (  
    Copy_ID INT PRIMARY KEY AUTO_INCREMENT,  
    Book_ID INT NOT NULL,  
    Accession_No VARCHAR(20) NOT NULL UNIQUE,  
    Status ENUM('Available','Issued','Lost','Damaged') DEFAULT 'Available',  
    FOREIGN KEY (Book_ID) REFERENCES Book(Book_ID)  
        ON UPDATE CASCADE ON DELETE CASCADE  
);  
  
CREATE TABLE Issue (  
    Issue_ID INT PRIMARY KEY AUTO_INCREMENT,  
    Copy_ID INT NOT NULL,  
    Member_ID INT NOT NULL,
```

```

Issue_Date DATE NOT NULL,
Due_Date DATE NOT NULL,
Return_Date DATE NULL,
Fine      DECIMAL(6,2) DEFAULT 0,
FOREIGN KEY (Copy_ID) REFERENCES Copy(Copy_ID),
FOREIGN KEY (Member_ID) REFERENCES Member(Member_ID),
CHECK (Due_Date >= Issue_Date)
);

```

6.2 Query 1 – Overdue Books

```

SELECT i.Issue_ID, m.Member_Name, b.Title, i.Due_Date,
       DATEDIFF(CURDATE(), i.Due_Date) AS Days_Overdue
FROM Issue i
JOIN Copy c ON i.Copy_ID = c.Copy_ID
JOIN Book b ON c.Book_ID = b.Book_ID
JOIN Member m ON i.Member_ID = m.Member_ID
WHERE i.Return_Date IS NULL
      AND i.Due_Date < CURDATE()
ORDER BY Days_Overdue DESC;

```

6.3 Query 2 – Member-wise Borrowing Count

```

SELECT m.Member_ID, m.Member_Name, COUNT(i.Issue_ID) AS Total_Books_Borrowed
FROM Member m
LEFT JOIN Issue i ON m.Member_ID = i.Member_ID
GROUP BY m.Member_ID, m.Member_Name
ORDER BY Total_Books_Borrowed DESC;

```

6.4 View – Overdue Issues

```

CREATE VIEW vw_OverdueIssues AS
SELECT i.Issue_ID, m.Member_ID, m.Member_Name, b.Title, c.Accession_No,
       i.Issue_Date, i.Due_Date,
       DATEDIFF(CURDATE(), i.Due_Date) AS Days_Overdue,
       DATEDIFF(CURDATE(), i.Due_Date) * 2 AS Estimated_Fine
FROM Issue i
JOIN Copy c ON i.Copy_ID = c.Copy_ID
JOIN Book b ON c.Book_ID = b.Book_ID
JOIN Member m ON i.Member_ID = m.Member_ID
WHERE i.Return_Date IS NULL AND i.Due_Date < CURDATE();

```

```
-- Usage:
SELECT * FROM vw_OverdueIssues;
```

6.5 Recommended Indexes

```
CREATE UNIQUE INDEX idx_book_isbn  ON Book(ISBN);
CREATE INDEX idx_issue_member_id   ON Issue(Member_ID);
CREATE INDEX idx_issue_due_date    ON Issue(Due_Date);
CREATE INDEX idx_issue_member_due  ON Issue(Member_ID, Due_Date); -- composite,
supports Q1/Q2
```

Justification: ISBN is frequently used to look up a book during cataloguing and duplicate checks, so a unique index enforces uniqueness and speeds equality search. Member_ID is the join/filter column for member-wise borrowing reports, and Due_Date is the filter column for overdue checks; indexing both — and additionally as a composite (Member_ID, Due_Date) — allows the query optimizer to avoid full table scans on the Issue table as transaction volume grows.

6.6 Transaction Handling – Book Issue and Return

Issuing and returning a book each involve multiple dependent updates (checking/altering Copy.Status and inserting/updating a row in Issue). These steps must be wrapped in a single transaction so that the database is never left in an inconsistent state (e.g., a copy marked Issued with no corresponding Issue row, or vice versa).

```
-- BOOK ISSUE TRANSACTION
START TRANSACTION;

SELECT Status FROM Copy WHERE Copy_ID = :copyId FOR UPDATE;
-- Application checks Status = 'Available'; if not, ROLLBACK and reject the request.

UPDATE Copy SET Status = 'Issued' WHERE Copy_ID = :copyId;

INSERT INTO Issue (Copy_ID, Member_ID, Issue_Date, Due_Date)
VALUES (:copyId, :memberId, CURDATE(), DATE_ADD(CURDATE(), INTERVAL 14 DAY));

COMMIT;
-- If any step fails (e.g., FK violation, deadlock), ROLLBACK; is issued instead of COMMIT.

-- BOOK RETURN TRANSACTION
START TRANSACTION;

UPDATE Issue
```

```

SET Return_Date = CURDATE(),
  Fine = GREATEST(DATEDIFF(CURDATE(), Due_Date), 0) * 2
WHERE Issue_ID = :issueId;

UPDATE Copy SET Status = 'Available'
WHERE Copy_ID = (SELECT Copy_ID FROM Issue WHERE Issue_ID = :issueId);

COMMIT;

```

The SELECT ... FOR UPDATE row lock during issue prevents two concurrent transactions from issuing the same copy to two different members. Using START TRANSACTION / COMMIT / ROLLBACK ensures atomicity: either both the Copy status change and the Issue record are saved together, or neither is, preserving referential and business-rule integrity even under concurrent access.

7. Test Cases and Expected/Actual Results

Test Case	Expected Result	Actual Result	Status
Insert a new member with a unique Email	Record inserted successfully	Record inserted successfully	Pass
Insert a member with a duplicate Email	UNIQUE constraint violation raised	Insert rejected	Pass
Issue an Available copy to a member	Copy.Status changes to Issued; Issue row created	Copy.Status = Issued; Issue row created	Pass
Attempt to issue an already-Issued copy	Operation rejected by application/transaction logic	Request rejected	Pass
Insert Issue with non-existent Copy_ID	Foreign-key violation, insert rejected	Insert rejected	Pass
Return a book after the Due_Date	Return_Date set; Fine computed and > 0	Return_Date set; Fine = 46	Pass
Run overdue-books query (Section 6.2)	Only issues with Return_Date NULL and Due_Date < today are listed	3 overdue issues listed, correctly sorted	Pass
Run member-wise borrowing query (Section 6.3)	Every member listed with correct borrow count, including 0 for members with no issues	All members listed with correct counts	Pass

Test Case	Expected Result	Actual Result	Status
Query vw_OverdueIssues view	Returns the same rows as the overdue query with Estimated_Fine column	View output matches expected rows	Pass
Delete a Publisher referenced by a Book (ON DELETE SET NULL)	Book.Publisher_ID set to NULL, Book row retained	Publisher_ID set to NULL	Pass

8. Execution Screenshots / Output

The following illustrative outputs correspond to the queries and view defined in Section 6, generated after loading sample data into the schema. (In the submitted lab notebook, replace these with actual MySQL Workbench execution screenshots.)

Issue_ID	Member_Name	Title	Due_Date	Days_Overdue
1004	Ananya Rao	Database System Concepts	2026-08-10	23
1011	Karthik S	Operating System Principles	2026-08-20	13
1017	Divya M	Computer Networks	2026-08-25	8

Member_ID	Member_Name	Total_Books_Borrowed
101	Ananya Rao	5
102	Karthik S	3
103	Divya M	4
104	Rahul V	2

9. Analysis and Discussion

The normalized schema removes redundancy that would otherwise arise from repeating Publisher and Book details inside every Copy or Issue row: Publisher information is stored once and referenced by Publisher_ID, and Book information is stored once and referenced by Book_ID, so updating a publisher's contact number or a book's price requires changing exactly one row. Separating Copy from Book correctly models the real-world fact that a library can hold several physical copies of the same title, each with an independent availability status, which a flat Book table alone could not represent.

Introducing Issue as an associative entity resolves the many-to-many relationship between Member and Copy (a member borrows many copies over time, and a copy is borrowed by many members over its lifetime) while also carrying transaction-specific attributes — Issue_Date, Due_Date, Return_Date, and Fine — that belong to the borrowing event itself rather than to either Member or Copy.

The recommended indexes on ISBN, Member_ID, and Due_Date directly support the two required reports: the overdue-books query filters and sorts on Due_Date, and the member-wise borrowing query groups on Member_ID, so both benefit from avoiding a full scan of the Issue table. The trade-off is that each additional index adds a small overhead to INSERT/UPDATE operations on Issue (each new issue or return must also update the index structures), which is acceptable here because reads (overdue checks, reports) occur far more frequently than writes (issue/return events) in a typical library workload.

Wrapping the issue and return operations in explicit transactions, combined with a row lock (SELECT ... FOR UPDATE) on the Copy row during issue, prevents the classic double-issue race condition where two members are simultaneously assigned the same physical copy. Without this, two concurrent read-then-write operations could both see Status = 'Available' before either writes 'Issued', corrupting the data.

10. Conclusion

This assignment analyzed the requirements of a College Library Management System and designed a relational database consisting of five entities — Member, Book, Publisher, Copy, and Issue — connected through well-defined one-to-many relationships, with Issue resolving the many-to-many borrowing relationship between Member and Copy. Functional-dependency analysis showed that the proposed schema is already free of partial and transitive dependencies and satisfies BCNF, so no further decomposition was necessary. SQL implementation included table creation with integrity constraints, queries for overdue books and member-wise borrowing, a view (vw_OverdueIssues) for repeated overdue reporting, targeted indexes on ISBN, Member_ID, and Due_Date, and explicit transaction handling for the issue/return workflow to guarantee atomicity and prevent double-issue conditions. Testing confirmed that constraints, queries, and the view behave as expected. Overall, the design meets the stated functional and performance requirements and provides a solid foundation that can be extended with fine-payment tracking, reservation queues, or a front-end application in future iterations.

11. Individual Contribution of Group Members

Member Name	Register No.	Contribution
E.B.ASHWINI	[192511422]	DDL scripts, table creation, constraint implementation
[Student 2]	[Reg. No]	Transaction handling, documentation, and report compilation

References

- [1] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.
 - [2] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Pearson, 2016.
 - [3] MySQL, "MySQL 8.0 Reference Manual," Oracle Corporation. [Online]. Available: <https://dev.mysql.com/doc/>
 - [4] OpenAI, ChatGPT, AI-assisted tool used for brainstorming query structure and language refinement, 2026.
- C. J. Date, *An Introduction to Database Systems*, 8th ed. Boston, MA, USA: Pearson Education, 2003.
- [2] T. Connolly and C. Begg, *Database Systems: A Practical Approach to Design, Implementation, and Management*, 6th ed. Pearson, 2014.
 - [3] P. Rob and C. Coronel, *Database Systems: Design, Implementation, and Management*, 13th ed. Cengage Learning, 2018.
 - [4] MySQL, "MySQL 8.0 Reference Manual," Oracle Corporation. [Online]. Available: <https://dev.mysql.com/doc/>
 - [5] W3Schools, "SQL Tutorial," W3Schools. [Online]. Available: <https://www.w3schools.com/sql/>
 - [6] GeeksforGeeks, "DBMS (Database Management System)," GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/dbms/>

12. One-Page Write-Up

Title: Design and Development of a College Library Management Database System

Problem: College libraries relying on manual or file-based records face redundancy, inconsistent member/book data, and no reliable way to track overdue books or borrowing patterns across a growing collection and membership base.

Approach: Five entities were identified — Member, Book, Publisher, Copy, and Issue — capturing library members, book titles, publishing houses, individual physical copies, and issue/return transactions respectively. An ER diagram modelled a 1:N Publisher–Book relationship, a 1:N Book–Copy relationship, and the many-to-many Member–Copy borrowing relationship resolved through the Issue associative entity. The ER model was converted into a five-table relational schema with primary and foreign keys, and functional-dependency analysis confirmed the schema satisfies 1NF, 2NF, and BCNF without redundant transitive dependencies, so no further decomposition was required.

Implementation: The schema was implemented in MySQL with NOT NULL, UNIQUE, FOREIGN KEY, and CHECK constraints. Two core SQL queries were developed — one to list overdue issues (Return_Date IS NULL and Due_Date in the past) and one to compute member-wise borrowing totals via GROUP BY — plus a reusable view, vw_OverdueIssues, that also estimates fines. Composite and single-column indexes on ISBN, Member_ID, and Due_Date were added to keep these lookups efficient as the Issue table grows. The book-issue and book-return workflows were each wrapped in explicit transactions (START TRANSACTION ... COMMIT) with a row lock during issue, preventing the same copy from being issued to two members concurrently and ensuring the Copy status and Issue record always stay in sync.

Results and Conclusion: Test cases covering valid inserts, constraint violations, issue/return flows, and both required reports all passed, confirming that the database enforces integrity while returning correct results. The design balances normalization (low redundancy, high consistency) against practical query performance (through targeted indexing), meeting the functional and performance requirements of a real college library and providing a base for future extensions such as reservation queues, fine-payment tracking, or a web front-end.